Network Security Automation & Audit Pipeline

A Python-based network security automation tool that connects to network devices, performs security audits, parses logs using NLP, generates HTML reports, and delivers automated email alerts — building an end-to-end detection and compliance pipeline.

Project Overview

This pipeline automates the process of auditing network device security configurations and analyzing logs for anomalies. It was built to demonstrate how Python can be used to enforce security baselines, detect suspicious activity, and reduce manual analyst workload through automation.

Key capabilities:

- Automated backup of running device configurations via SSH
- Security compliance auditing against a defined baseline
- Log ingestion and NLP-based event classification
- Structured HTML report generation
- Automated email delivery of audit results

Architecture

```
devices.csv
    |
    ▼
main.py (Orchestrator)
    |
    ├── backup.py       → SSH connection, config backup, log retrieval
    ├── audit.py        → Security baseline compliance checks
    ├── log_parser.py   → Regex + NLP log parsing and classification
    ├── report.py       → HTML report generation
    ├── emailer.py      → Automated email delivery via SMTP
    ├── logger.py       → Application logging
    └── configuration.py → Environment variables and CSV reader
```

Features

1. Device Connectivity & Backup

- Connects to network devices using **Netmiko** over SSH
- Retrieves running configuration and saves timestamped backups
- Supports Cisco IOS-XR and other Netmiko-compatible device types

2. Security Compliance Audit

Checks each device configuration against the following security baseline:

Check	Description
SSH	SSH server configured on default VRF
AAA Authentication	AAA login authentication enabled
Login Banner	MOTD banner configured
NTP	NTP server configured
Password Encryption	Type 10 (SHA-256) password encryption
SNMP	SNMP community string configured

3. Log Parsing & NLP Classification

- Parses raw device logs using **regex patterns** for timestamp, severity, mnemonic, and message extraction
- Uses **NLTK tokenization** to classify log events into categories:
  - `authentication_failure` — permission denied, PAM failures, failed logins
  - `rate_limit_exceeded` — rate limiting events
  - `connection_event` — connection closed/opened events
  - `successful_login` — accepted authentication events
  - `unknown` — unclassified events

4. HTML Report Generation

Generates a structured HTML report per device including:

- Security compliance check results

- Total log events analyzed
- Authentication failure count
- Rate limit exceeded count
- Connection events count
- Successful login count

5. Automated Email Alerts

- Delivers HTML audit reports via **Gmail SMTP**
- Uses TLS encryption for secure email transmission
- Configurable sender, receiver, and credentials via environment variables

---

Project Structure

```
network-security-pipeline/
│
├── main.py              # Orchestrator — runs full pipeline
├── backup.py            # Device connection, backup, log retrieval
├── audit.py             # Security baseline compliance checks
├── log_parser.py        # Log parsing and NLP classification
├── report.py            # HTML report generation
├── emailer.py           # Email delivery
├── logger.py            # Application logging
├── configuration.py     # Environment variables and CSV reader
│
├── devices.csv          # Device inventory
├── .env                 # Credentials (not committed to Git)
├── .gitignore           # Excludes .env, backups, logs
│
├── backups/             # Timestamped device config backups
├── logs/                # Application logs
└── reports/             # Generated HTML reports
```

Setup & Installation

Prerequisites

- Python 3.8+
- Network device accessible via SSH
- Gmail account with App Password enabled

Install Dependencies

```bash
pip install netmiko python-dotenv nltk
python -c "import nltk; nltk.download('punkt')"
```

Configure Environment Variables

Create a `.env` file in the project root:

```env
NET_USERNAME=your_device_username
NET_PASSWORD=your_device_password
NET_SECRET=your_enable_secret

EMAIL_SENDER=your_email@gmail.com
EMAIL_PASSWORD=your_gmail_app_password
EMAIL_RECEIVER=recipient@email.com
```

> **Security Note:** Never commit `.env` to version control. Add it to `.gitignore`.

Configure Devices

Edit `devices.csv` to add your network devices:

```csv
hostname,ip,device_type,site,criticality
router1,192.168.1.1,cisco_ios,HQ,high
switch1,192.168.1.2,cisco_ios,Branch,medium
```

Run the Pipeline

```bash
```

```
python main.py
```

Sample Output

Audit Report Email:

```
Device: devnetsandboxiosxe.cisco.com

Security Checks:
ssh: True
aaa: True
login_banner: False ← FAILED
ntp: True
password_encryption: True
snmp: False ← FAILED

Log Analysis:
Total Events: 147
Authentication Failures: 12
Rate Limits Exceeded: 3
Connection Events: 28
Successful Logins: 104
```

Security Considerations

- All credentials stored in `.env` file — never hardcoded
- SSH used for all device connections
- Email transmission uses TLS encryption
- Backups stored locally with timestamps for audit trail
- Application logs maintained for troubleshooting

MITRE ATT&CK Relevance

This pipeline detects and documents activity relevant to:

Technique	ID	Detection
Brute Force	T1110	Authentication failure counting
Valid Accounts	T1078	Successful login tracking
Network Device CLI	T1059.008	Configuration auditing
Credentials in Files	T1552.001	Password encryption checks

Technologies Used

Technology	Purpose
Python	Core language
Netmiko	Network device SSH automation
NLTK	Natural language log classification
Regex	Log pattern extraction
python-dotenv	Secure credential management
smtplib	Email automation
CSV	Device inventory management
Cisco DevNet Sandbox	Testing environment

Author

Georgina Maima
M.S. Computer Science (Cybersecurity) — Boston University
LinkedIn | GitHub

Future Improvements

- Add support for more device types (Juniper, Palo Alto)

- Integrate with SIEM platforms via syslog forwarding
- Add dashboard visualization for audit trends
- Implement severity-based alerting thresholds
- Add CVE checking against device software versions